张 曙
ZHANG SHU

He/him | Age: 27 | +86 13851778481 | evianzhang1999@gmail.com

Evian-Zhang EvianZhang

Job Target: Open-Source Community Maintainer
(Rust and/or Security)



EDUCATION

ISCAS	**Institute of Software Chinese Academy of Sciences**	PhD	Cybersecurity Mentor: Purui Su	2021 — 2027
	Southeast University	Bachelor	Cybersecurity	2017 — 2021

ACHIEVEMENT

Paper list

Zhang S, et al. In-Memory Fuzzing for Server-Side COTS Applications[J]. Journal of Cyber Security. 2026

Zhang S, et al. JanuSafe: Detecting Cross-Language Semantic Violations in Rust FFI Wrappers for C/C++ Libraries[C]. ICSE, 2027. (Under review)

Zhang S, et al. Gone with the Unwind: Detecting Exception Safety Violations in Rust[C]. SANER, 2027. (Under review)

Software Copyrights & Patents

Pathfinder Vulnerability Mining System V1.0

Function-Level Fuzzing Method and System for Server-Side Applications

A Core-Binding Strategy for Fuzzing

Dynamic Instruction Emulation Execution Extension and Verification Scheme

Virtual Network Topology Construction Method

Binary-Level Runtime Function Boundary Identification Method

Vulnerabilities

Multiple high-risk memory vulnerabilities in closed-source COTS server-side applications

30+ remote code injection / path traversal vulnerabilities in mainstream open-source Web applications

80+ vulnerabilities in Rust libraries with 8.1B+ downloads in total

SKILL

Language

7-year Rust development experience; proficient in Rust and C; skilled in C++, Python, Go, x86 & AArch64 assembly, TypeScript.

Tool & AI

Linux operating system, Docker, Git collaborative development, GitHub community maintenance.

Software development and vulnerability mining with AI agents including Claude-Code, OpenCode, PI.

English

Highly proficient in professional English for literature reading and academic communication. CET-6 certified.

Virtualization

Familiar with QEMU/KVM frameworks and Linux virtual networking.

Vulnerability Detection & Program Analysis

Experienced with fuzzing frameworks such as AFL++, LibAFL; execution engines including Unicorn, QEMU-usermode, DynamoRIO; static analysis tools such as LLVM IR and IDA. Capable of memory vulnerability mining for binaries and C/C++ source code, as well as taint-style vulnerability mining for scripting languages.

Software Development

Back-end development in Rust with Axum and Tokio; MCP server implementation based on rmcp; Front-end development in TypeScript with React and Next.js.

PROJECT

I am core contributor to multiple major internal projects with strong teamwork capabilities.

Agent Framework for Vulnerability Detection (Python)

Enhances multi-agent collaboration, sandbox execution, and program static-dynamic analysis capabilities for agent back-ends such as Claude-Code and Deepseek Harness. Provides migration solutions for open-source security agents including codex-security.

Function-Level Fuzzing (Rust)

Attack-surface identification and function-level memory-fuzzing capabilities for network-service programs on top of AFL++ and Unicorn. Achieves 1500× performance compared with AFLNET. Integrated into the vulnerability-detection agent framework.

Dynamic Instruction Trace Analysis Suite SDK (Rust)

Provides high-level dynamic analysis for instruction traces recorded from hypervisors, user-space contexts and function-level snippets. Features include trace parsing & search, runtime function-boundary recognition, call-stack reconstruction and call-tree recovery. Exposes C and Python SDKs as well as MCP interfaces. Has been stably deployed in core internal projects for three years.

Binary-Instruction-Level Tracing (Rust)

Implements a generic binary instruction-level tracing backend using multi-buffer rotation strategies. Integrated with QEMU-usermode and Unicorn engines to support on-demand tracing of user-space programs and function-level code fragments. Delivers 5× speed-up over legacy internal tools.

Virtual Network Topology Construction System (Rust)

Implements a declarative, overlappable virtual-network topology system built upon Linux network namespaces, optimized for QEMU/KVM workloads.

Command-Injection/Path-Traversal Vulnerability Detection for Web Applications (Python)

Uses RAG for full-program code indexing to improve semantic retrieval for agents. Performs vulnerability detection following taint-style analysis strategies. (Under research)

Misc

System-level forced-execution system; KVM-based Windows VM introspection & instruction trace recording system; non-intrusive Nyx system-mode fuzzing tool, etc.

OPENSOURCE INVOLVEMENT

Created

llvm-ir-tutorial 🔗(1.5K+ Stars)	LLVM IR Tutorial
learn-assembly-on-Apple-Silicon-Mac 🔗	AArch64 Assembly Tutorial
wasm-tutorial 🔗	WASM Tutorial
Assembly-on-macOS 🔗	x86-64 Tutorial
Awesome Rust Papers 🔗	Collection of High-Quality Rust Papers
static-keys 🔗(87K+ Downloads)	User-mode runtime instruction patching (W/M/L, x86/x86-64/AArch64)
iptr 🔗	Intel PT Decoder

Participated

LibAFL 🔗	General Fuzzing Framework in Rust
Unicorn 🔗	Instruction Emulation Engine
UnicornAFL 🔗	Unicorn-based Fuzzing Framework
mdBook 🔗	Static Documentation Site Generation in Rust
rmcp 🔗	Official MCP Framework in Rust